

Reinforcement Learning

n -step Bootstrapping Sutton/Barto* Chapter 7

Michael Hahsler, SMU

With figures from Sutton/Barto*

*Sutton and Barto, Reinforcement Learning: An Introduction,
2nd edition, MIT Press, Cambridge, MA, 2018



Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\mathbf{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\mathbf{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bold)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X , i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\operatorname{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or q_*
$\bar{V}_t(s)$	expected approximate action value
U_t	target for estimate at time t

MDP

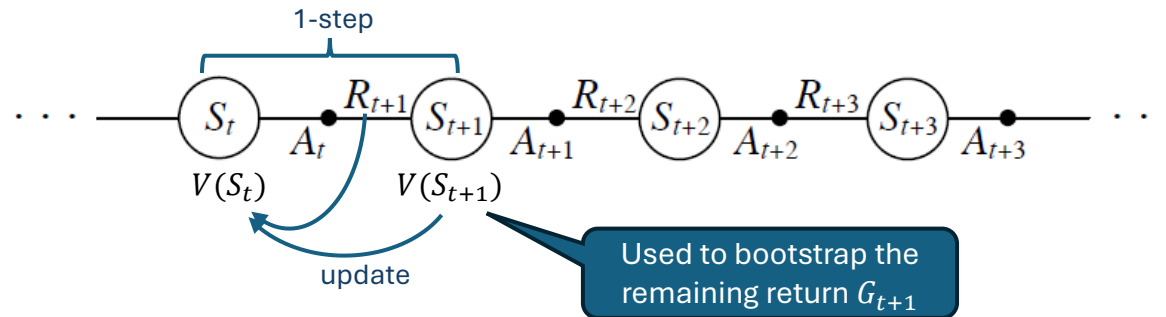
s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r s, a)$	probability of transition to state s' and receiving reward r , from state s taking action a
	expected immediate reward from state s after action a
$\pi(a s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

1-step TD vs. n -step TD vs. MC

How far do we look in the sampled episode ahead to update the state value?

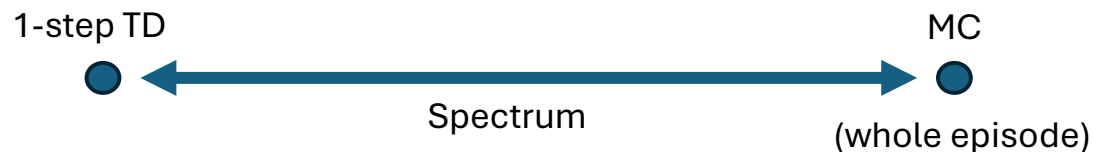
1-step DT

- We have already discussed 1-step TD: Q-Learning, Sarsa
- We only observe the next step and its reward. The remaining return is bootstrapped using the estimate of the next state's value $V(S_{t+1})$ or action values $Q(S_{t+1}, \cdot)$.



n -step DT

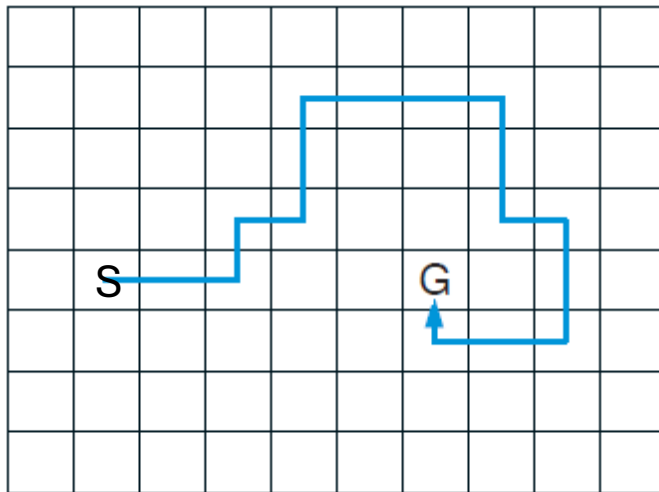
Looks n time steps ahead and bootstraps the rest.
TD unifies and generalizes MC and one-step TD



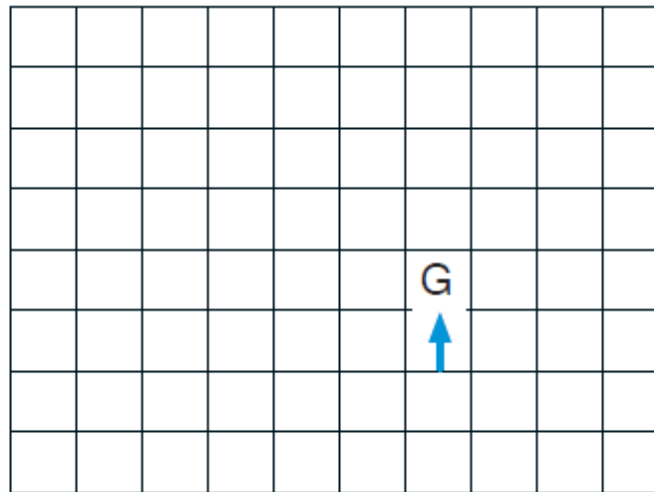
Why is a Longer Look-ahead Useful?

- **Example:** Maze with only one final reward for reaching the goal. We learn from the first episode.

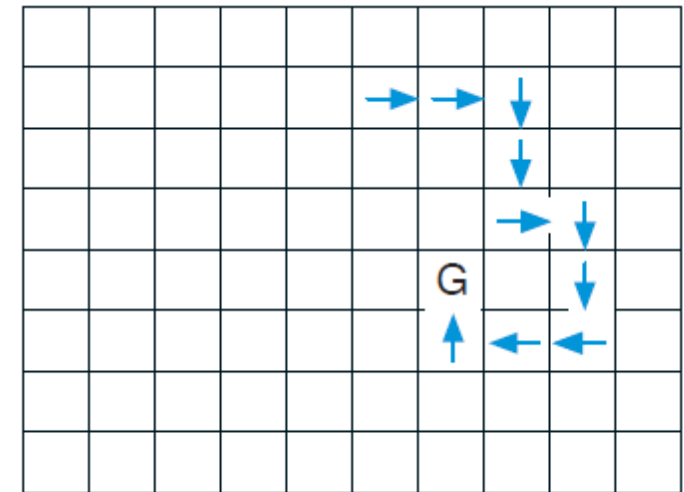
Episode (MC)



1-step look ahead



10-step look ahead

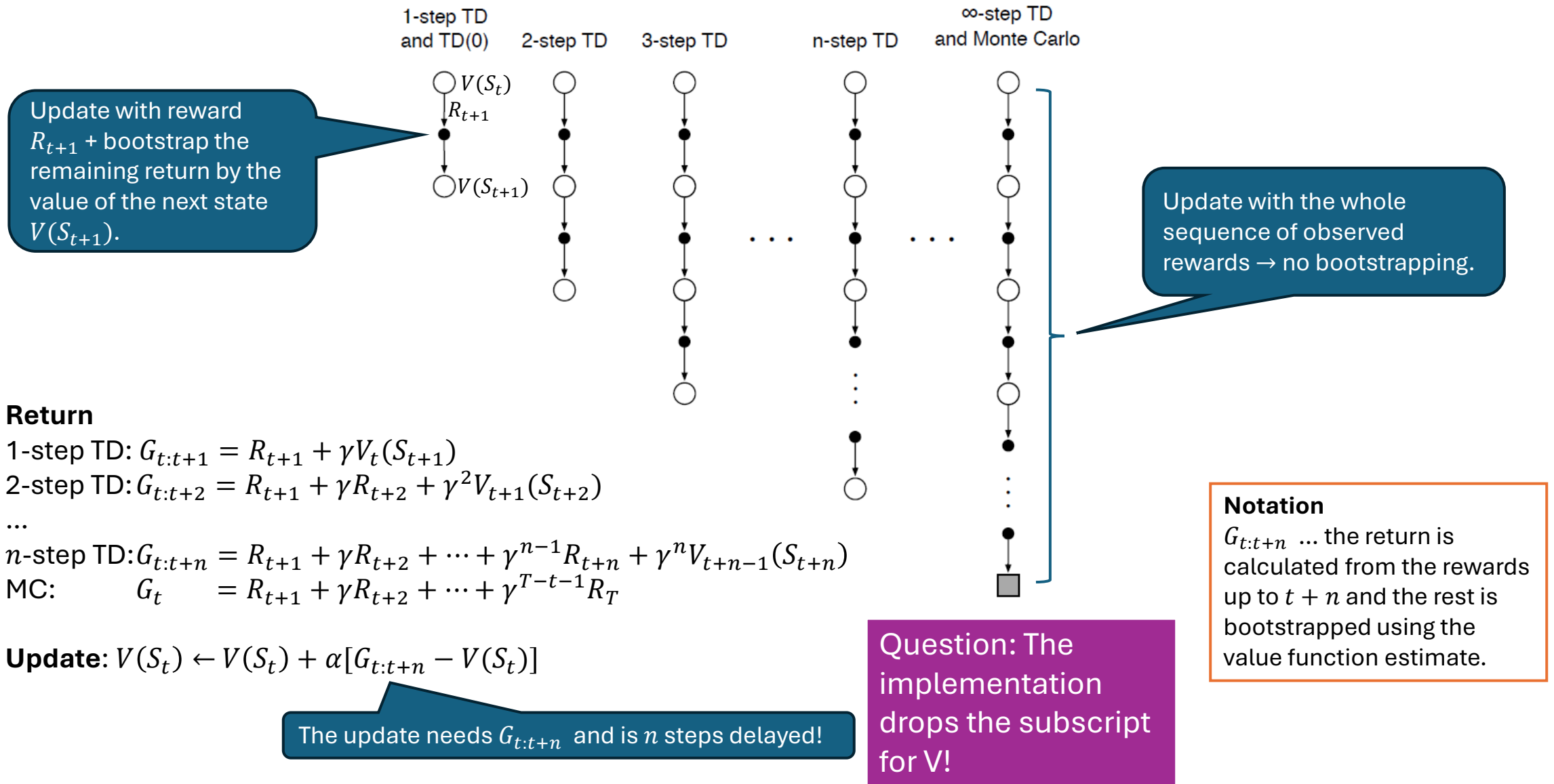


Updates 10 state values with the goal reward.
Learns faster.

n -step TD Prediction

Estimate the value function for a given policy.

n -step TD Prediction



Alg. n -step TD Prediction

n -step TD for estimating $V \approx v_\pi$

Input: a policy π

Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n

Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$

All store and access operations (for S_t and R_t) can take their index mod $n + 1$

Loop for each episode:

Initialize and store $S_0 \neq \text{terminal}$

$T \leftarrow \infty$

Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take an action according to $\pi(\cdot|S_t)$

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then $T \leftarrow t + 1$

$\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)

 If $\tau \geq 0$:

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

 If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$

$V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$

 Until $\tau = T - 1$

end of episode is
detected below

start only when we
can update $t \geq n$

The update

Use bootstrapping if the
episode is not over

Convergence

Error Reduction Property of n -step Returns

- Uses V_{t+n-1} to correct for the missing rewards after R_{t+n}
- The worst error of the expected n -step return is guaranteed to be less than or equal to the discounted the worst error under V_{t+n-1}

$$\max_s |\mathbb{E}_\pi[G_{t:t+n} | S_t = s] - v_\pi(s)| \leq \gamma^n \max_s |V_{t+n-1}(s) - v_\pi(s)|$$

→ the update is guaranteed to reduce the error meaning that all n -step methods (including 1-step and MC) **converge**!

n -step Control (Sarsa)

Learn a policy. This is done by estimating Q-values instead of V-values.

n -step Sarsa

Note (0) has
nothing to do with
the steps!

- Original Sarsa is called one-step Sarsa or Sarsa(0)

n -step Sarsa

- On policy
- Remember: The policy needs to keep exploring so we learn an ϵ -greedy policy
- Use state-action pairs and learns Q so we can find the ϵ -greedy policy
- Return (redefined using Q):
$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q_{t+n-1}(S_{t+n}, A_{t+n})$$
- Update rule:
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_{t:t+n} - Q(S_t, A_t)]$$

Speedup of n -step Sarsa

Q -values updated during one episode

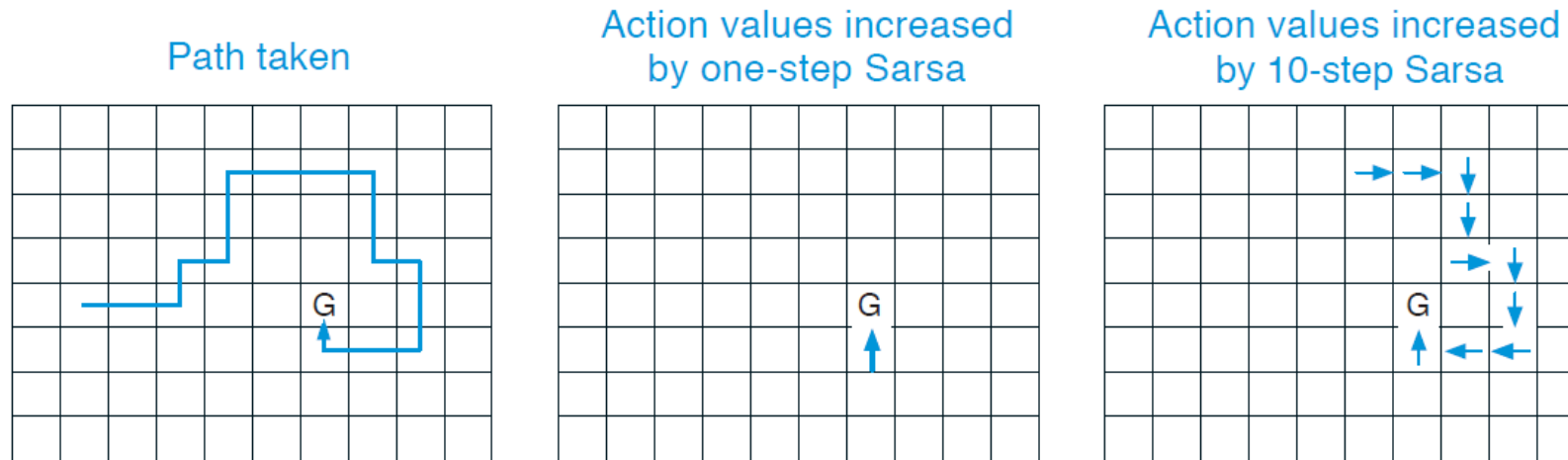


Figure 7.4: Gridworld example of the speedup of policy learning due to the use of n -step methods. The first panel shows the path taken by an agent in a single episode, ending at a location of high reward, marked by the G. In this example the values were all initially 0, and all rewards were zero except for a positive reward at G. The arrows in the other two panels show which action values were strengthened as a result of this path by one-step and n -step Sarsa methods. The one-step method strengthens only the last action of the sequence of actions that led to the high reward, whereas the n -step method strengthens the last n actions of the sequence, so that much more is learned from the one episode.

Alg. From DT prediction to n -step Sarsa Control

π is given

n -step TD for estimating $V \approx v_\pi$

Input: a policy π
 Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n
 Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
 All store and access operations (for S_t and R_t) can take their index mod $n + 1$

Loop for each episode:
 Initialize and store $S_0 \neq \text{terminal}$
 $T \leftarrow \infty$
 Loop for $t = 0, 1, 2, \dots$:
 | If $t < T$, then:
 | | Take an action according to $\pi(\cdot|S_t)$
 | | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
 | | If S_{t+1} is terminal, then $T \leftarrow t + 1$
 | $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
 | If $\tau \geq 0$:
 | | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
 | | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ ($G_{\tau:\tau+n}$)
 | | $V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$
 Until $\tau = T - 1$

Update V

n -step Sarsa for estimating $Q \approx q_\pi$ or q_*

Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
 Initialize π to be ϵ -greedy with respect to Q , or to a fixed given policy
 Algorithm parameters: step size $\alpha \in (0, 1]$, small $\epsilon > 0$, a positive integer n
 All store and access operations (for S_t , A_t , and R_t) can take their index mod $n + 1$

Loop for each episode:
 Initialize and store $S_0 \neq \text{terminal}$
 Select and store an action $A_0 \sim \pi(\cdot|S_0)$
 $T \leftarrow \infty$
 Loop for $t = 0, 1, 2, \dots$:
 | If $t < T$, then:
 | | Take action A_t
 | | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
 | | If S_{t+1} is terminal, then:
 | | | $T \leftarrow t + 1$
 | | else:
 | | | Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$
 | $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
 | If $\tau \geq 0$:
 | | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
 | | If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)
 | | $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$
 | | If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is ϵ -greedy wrt Q
 Until $\tau = T - 1$

Select next action
from the current π

Update Q and π

n -step Off-Policy Learning

n -step Off-Policy Learning

- **Goal:** Learn target policy π using behavioral policy b .
- **Method:** Use importance sampling ratio as a weight for updates

$$V_{t+1}(S_t) = V_{t+n-1}(S_t) + \alpha \rho_{t:t+n-1} [G_{t:t+n} - V_{t+n-1}(S_t)]$$

where the weight is

$$\rho_{t:h} = \prod_{k=t}^{\min(h, T-t)} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}$$

Probability that b chooses the same actions as π

- Can be used in n -step TD or n -step Sarsa.
- Note: Since the behavioral policy provides exploration, we can learn the best (greedy) policy.

Alg. Comparing On-Policy with Off-Policy

n -step Sarsa for estimating $Q \approx q_*$ or q_π

Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
 Initialize π to be ϵ -greedy with respect to Q , or to a fixed given policy
 Algorithm parameters: step size $\alpha \in (0, 1]$, small $\epsilon > 0$, a positive integer n
 All store and access operations (for S_t , A_t , and R_t) can take their index mod $n + 1$

Loop for each episode:

- Initialize and store $S_0 \neq \text{terminal}$
- Select and store an action $A_0 \sim \pi(\cdot | S_0)$
- $T \leftarrow \infty$
- Loop for $t = 0, 1, 2, \dots$:
 - If $t < T$, then:
 - Take action A_t
 - Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
 - If S_{t+1} is terminal, then:
 - $T \leftarrow t + 1$
 - else:
 - Select and store an action $A_{t+1} \sim \pi(\cdot | S_{t+1})$
 - $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
 - If $\tau \geq 0$:
 - $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
 - If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)
 - $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$
 - If π is being learned, then ensure that $\pi(\cdot | S_\tau)$ is ϵ -greedy wrt Q

Until $\tau = T - 1$

Off-policy n -step Sarsa for estimating $Q \approx q_*$ or q_π

Input: an arbitrary behavior policy b such that $b(a|s) > 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}$
 Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
 Initialize π to be greedy with respect to Q , or as a fixed given policy
 Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n
 All store and access operations (for S_t , A_t , and R_t) can take their index mod $n + 1$

Loop for each episode:

- Initialize and store $S_0 \neq \text{terminal}$
- Select and store an action $A_0 \sim b(\cdot | S_0)$
- $T \leftarrow \infty$
- Loop for $t = 0, 1, 2, \dots$:
 - If $t < T$, then:
 - Take action A_t
 - Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
 - If S_{t+1} is terminal, then:
 - $T \leftarrow t + 1$
 - else:
 - Select and store an action $A_{t+1} \sim b(\cdot | S_{t+1})$
 - $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
 - If $\tau \geq 0$:
 - $\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n, T-1)} \frac{\pi(A_i | S_i)}{b(A_i | S_i)}$
 - $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
 - If $\tau + n < T$, then: $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)
 - $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \rho [G - Q(S_\tau, A_\tau)]$
 - If π is being learned, then ensure that $\pi(\cdot | S_\tau)$ is greedy wrt Q

Until $\tau = T - 1$

Add behavior policy

Greedy target policy

Importance sampling ratio

The behavioral policy provides exploration so we can learn a greedy policy!

What you Need to Know

- n -step methods let us vary the learning behavior between on-step DT methods and Monte Carlo Methods (whole episode).
- Advantage:
 - **Learning:** Looking more steps ahead speeds up learning, especially for sparse rewards.
 - An intermediate amount of bootstrapping typically performs better than the one-step and MC extremes. n is application-dependent and can be tuned.
- Drawback:
 - **Delay:** Updates are delayed n -step since they need to know future events (actions and states).
 - **Computation:** More computation per time step and higher memory needs than one-step methods.

Eligibility traces address the delay and computation drawbacks. We will cover these methods later.